CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

ESCOLA SUPERIOR POLITÉCNICA

NOME DO CURSO

DESENVOLVIMENTO WEB BACK END

ATIVIDADE PRÁTICA
MODELO

MARCILEI MADUREIRA VELOSO – RU: 4980227

GOIÂNIA – GO

2026

ATIVIDADE PRÁTICA

1. **Descrição de uma situação fictícia**
2. **Diagrama caso de uso**
3. **Especificação da API desenvolvida**
4. **Prints do Postman**
5. **Link do repositório (GitHub ou similar) contendo o projeto(código fonte)**

1. Descrição de uma situação fictícia

A Baozi Store é uma pequena loja especializada na venda de pãozinho chinês. Para melhorar a organização do negócio, foi desenvolvido um sistema simples utilizando API REST para controlar clientes, produtos e pedidos.

Um cliente chamado **Marcilei4980227** realizou seu cadastro no sistema da loja.

O produto vendido pela loja chama-se **Baozi Tradicional de Frango**, vendido por unidade no valor de R$ 12,50.

Em determinado momento, o cliente realizou um pedido de 3 unidades do produto.

O sistema registra os dados do cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja e a organização das vendas.

---

2. Diagrama de Caso de Uso

Ator Principal

1. Usuário da API

Casos de Uso

Cliente

4 Cadastrar Cliente
5 Listar Clientes
6 Consultar Cliente por ID
7 Excluir Cliente

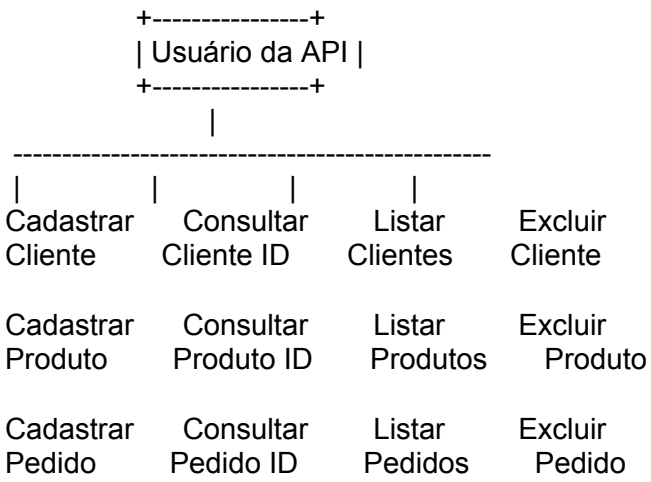
Produto

3 Cadastrar Produto
4 Listar Produtos
5 Consultar Produto por ID
6 Excluir Produto

Pedido

1 Cadastrar Pedido
2 Listar Pedidos
3 Consultar Pedido por ID
4 Excluir Pedido

Representação textual do diagrama UML



---

3. Especificação da API Desenvolvida

Tecnologias Utilizadas

- Java
- Spring Boot
- Spring Data JPA
- MySQL
- Postman
- Maven

Estrutura do Projeto

```
src/main/java
├── controller
├── model
├── repository
└── BaoziStoreApplication.java
```

Entidades Criadas

Entidade Produto

Campo	Tipo
id	Long
nome	String
preco	BigDecimal
estoque	Boolean

Exemplo JSON


```
{
  "nome": "Baozi Tradicional de Frango",
  "preco": 12.50,
  "estoque": true
}
```

Entidade Cliente

Campo	Tipo
id	Long
nome	String
clienteDesde	LocalDate

Exemplo JSON


```
{
  "nome": "Marcilei4980227",
  "clienteDesde": "2026-05-10"
}
```

Entidade Pedido

Campo	Tipo
id	Long
clienteId	Long
produtoId	Long
quantidade	Integer

Exemplo JSON


```
{
  "clienteId": 1,
  "produtoId": 1,
  "quantidade": 3
}
```

Endpoints Implementados

Cliente

Método	Endpoint	Função
POST	/clientes	Criar cliente
GET	/clientes	Listar clientes
GET	/clientes/{id}	Buscar cliente por ID
DELETE	/clientes/{id}	Excluir cliente

Produto

Método	Endpoint	Função
POST	/produtos	Criar produto
GET	/produtos	Listar produtos
GET	/produtos/{id}	Buscar produto por ID
DELETE	/produtos/{id}	Excluir produto

Pedido

Método	Endpoint	Função
POST	/pedidos	Criar pedido
GET	/pedidos	Listar pedidos
GET	/pedidos/{id}	Buscar pedido por ID
DELETE	/pedidos/{id}	Excluir pedido

---

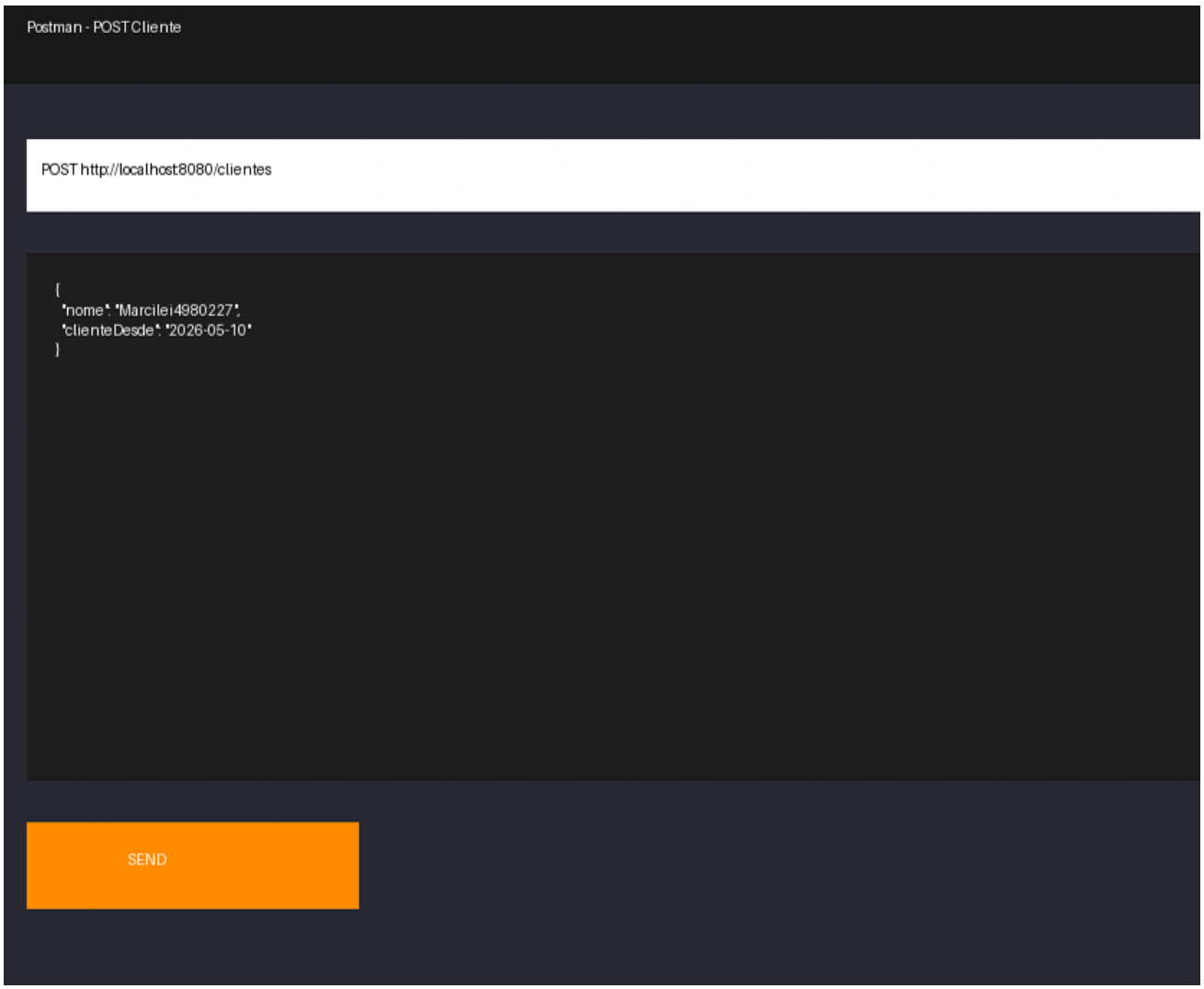
4. Prints do Postman

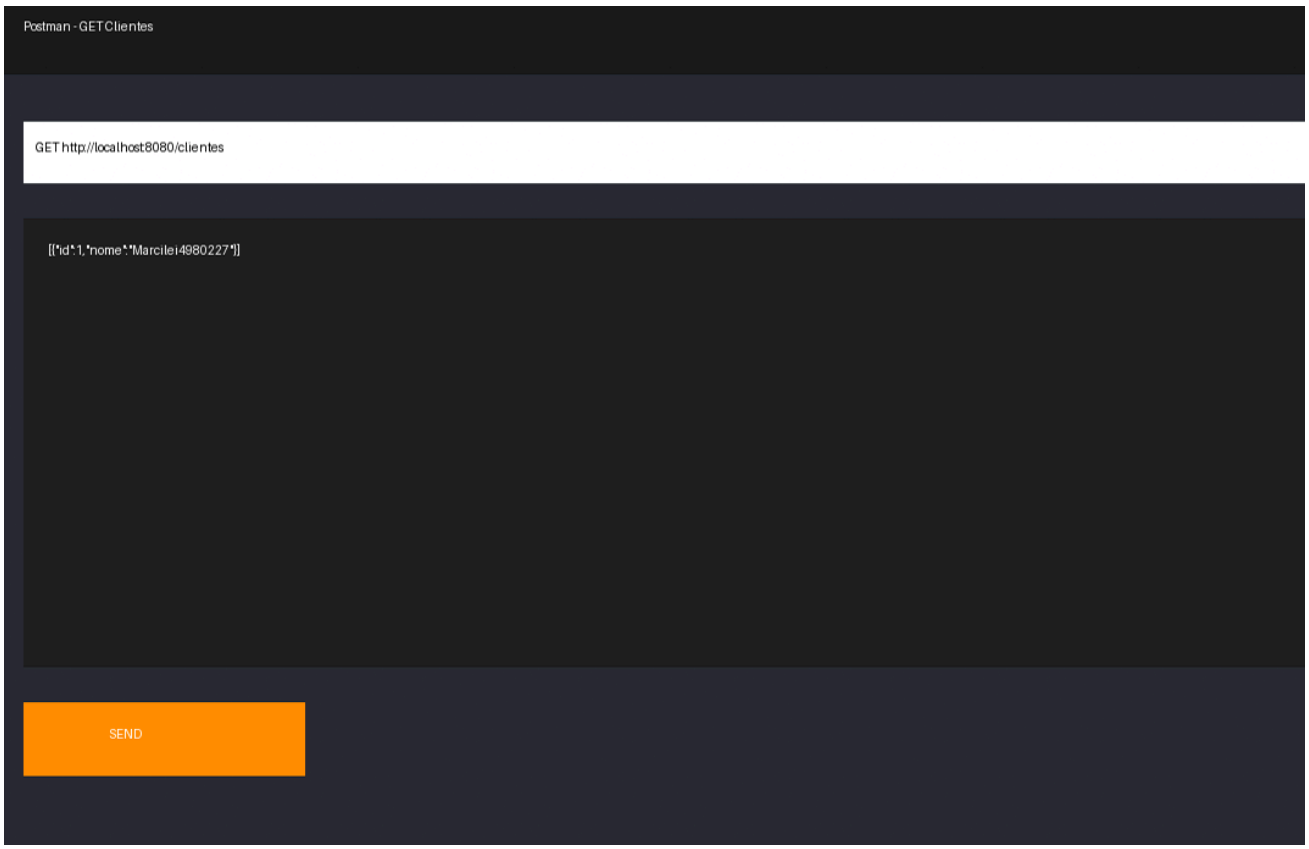
Prints obrigatórios

Inserir os prints abaixo:

Cliente

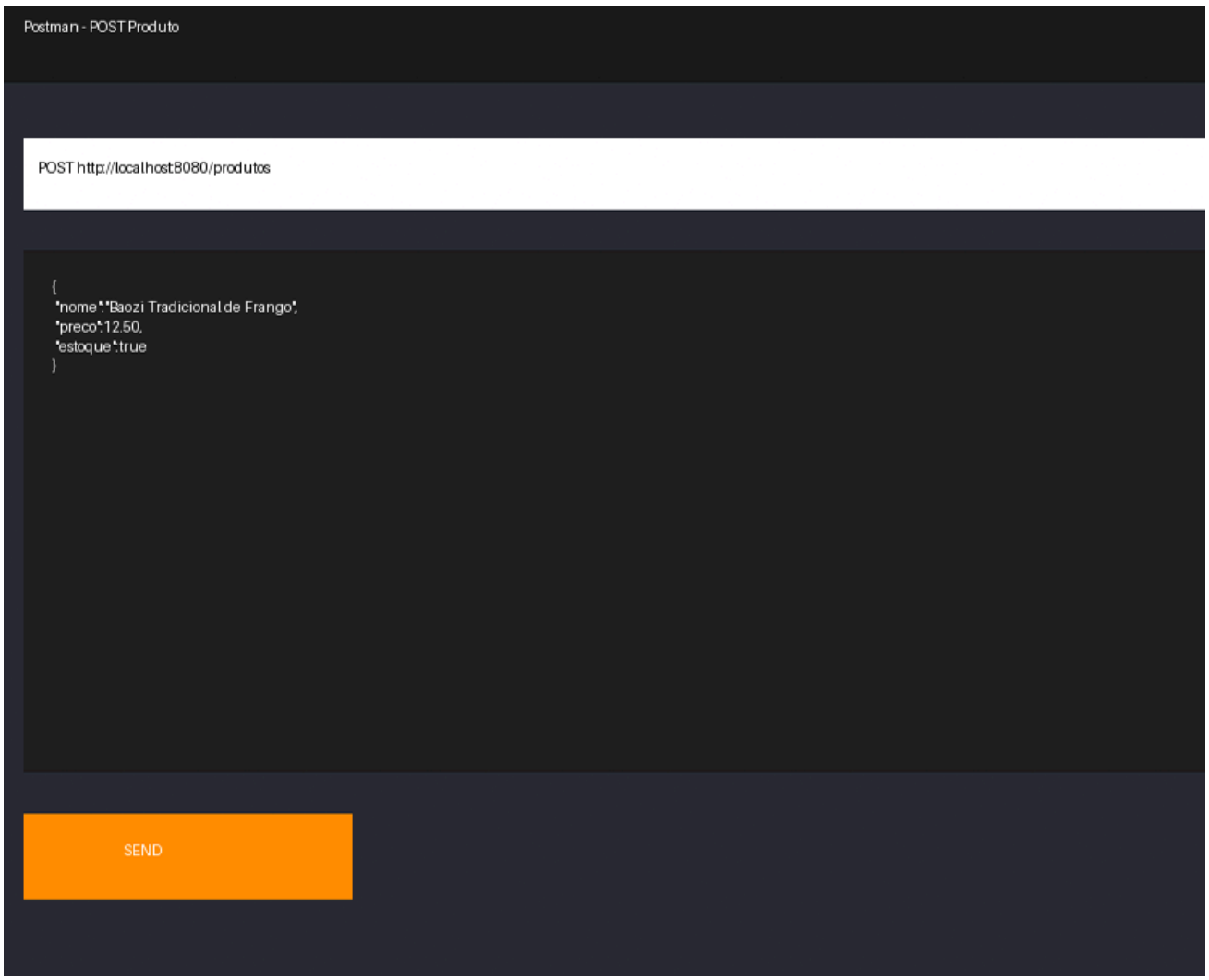
a) POST criando cliente
b) GET listando clientes
c) GET cliente por ID
d) DELETE cliente


Postman - POST Cliente
POST http://localhost:8080/clientes
{
"nome": "Marcilei4980227",
"clienteDesde": "2026-05-10"
}
SEND



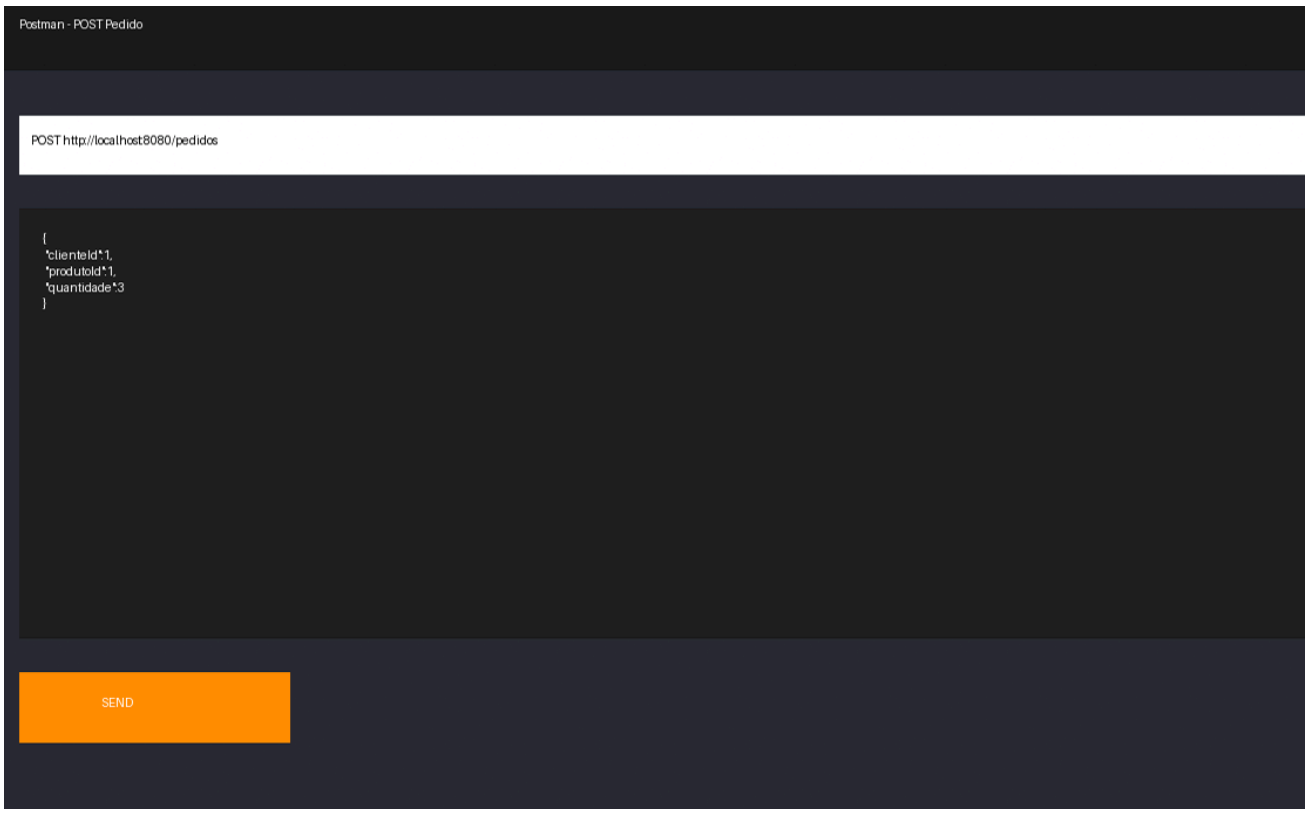
Produto

i. POST criando produto
ii. GET listando produtos
iii. GET produto por ID
iv. DELETE produto



Pedido

1- POST criando pedido
2- GET listando pedidos
3- GET pedido por ID
4- DELETE pedido



Exemplos para os prints do Postman

POST Cliente

```
{
  "nome": "Marcilei4980227",
  "clienteDesde": "2026-05-10"
}
```

POST Produto

```
{
  "nome": "Baozi Tradicional de Frango",
  "preco": 12.50,
  "estoque": true
}
```

POST Pedido

```
{
  "clienteId": 1,
  "produtoId": 1,
  "quantidade": 3
}
```

---

5. Link do Repositório

Link do GitHub contendo o código-fonte do projeto:

https://github.com/marcileiveloso-collab/baozi-store-api

Exemplo de Código das Entidades

Produto.java

```java
package model;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private BigDecimal preco;
    private Boolean estoque;

    public Produto() {}

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public BigDecimal getPreco() {
```

```
        return preco;
    }

    public void setPreco(BigDecimal preco) {
        this.preco = preco;
    }

    public Boolean getEstoque() {
        return estoque;
    }

    public void setEstoque(Boolean estoque) {
        this.estoque = estoque;
    }
}
```

Cliente.java

```
package model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private LocalDate clienteDesde;

    public Cliente() {}

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public LocalDate getClienteDesde() {
```

```
        return clienteDesde;
    }

    public void setClienteDesde(LocalDate clienteDesde) {
        this.clienteDesde = clienteDesde;
    }
}
```

Pedido.java

```
package model;

import jakarta.persistence.*;

@Entity
public class Pedido {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long clienteId;
    private Long produtoId;
    private Integer quantidade;

    public Pedido() {}

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Long getClienteId() {
        return clienteId;
    }

    public void setClienteId(Long clienteId) {
        this.clienteId = clienteId;
    }

    public Long getProdutoId() {
        return produtoId;
    }

    public void setProdutoId(Long produtoId) {
        this.produtoId = produtoId;
    }

    public Integer getQuantidade() {
```

```
        return quantidade;
    }

    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
}
```

Exemplo de Repository

```
package repository;

import model.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
}
```

Exemplo de Controller

```
package controller;

import model.Cliente;
import repository.ClienteRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/clientes")
public class ClienteController {

    private final ClienteRepository repository;

    public ClienteController(ClienteRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Cliente salvar(@RequestBody Cliente cliente) {
        return repository.save(cliente);
    }

    @GetMapping
    public List<Cliente> listar() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
```

```
    public Cliente buscar(@PathVariable Long id) {
        return repository.findById(id).orElse(null);
    }

    @DeleteMapping("/{id}")
    public void deletar(@PathVariable Long id) {
        repository.deleteById(id);
    }
}
```

---

application.properties

```
spring.application.name=baozi-store

spring.datasource.url=jdbc:mysql://localhost:3306/baozi_store
spring.datasource.username=root
spring.datasource.password=123456

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
```

Classe Principal

BaoziStoreApplication.java

```
package com.baozistore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BaoziStoreApplication {

    public static void main(String[] args) {
        SpringApplication.run(BaoziStoreApplication.class, args);
    }
}
```

Repositories

ProdutoRepository.java

```
package repository;

import model.Produto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProdutoRepository extends JpaRepository<Produto, Long> {
}
```

PedidoRepository.java

```
package repository;

import model.Pedido;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PedidoRepository extends JpaRepository<Pedido, Long> {
}
```

Controllers

ProdutoController.java

```
package controller;

import model.Produto;
import repository.ProdutoRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoRepository repository;

    public ProdutoController(ProdutoRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Produto salvar(@RequestBody Produto produto) {
        return repository.save(produto);
    }
```

```
    @GetMapping
    public List<Produto> listar() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
    public Produto buscar(@PathVariable Long id) {
        return repository.findById(id).orElse(null);
    }

    @DeleteMapping("/{id}")
    public void deletar(@PathVariable Long id) {
        repository.deleteById(id);
    }
}
```

PedidoController.java

```
package controller;

import model.Pedido;
import repository.PedidoRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/pedidos")
public class PedidoController {

    private final PedidoRepository repository;

    public PedidoController(PedidoRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Pedido salvar(@RequestBody Pedido pedido) {
        return repository.save(pedido);
    }

    @GetMapping
    public List<Pedido> listar() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
    public Pedido buscar(@PathVariable Long id) {
        return repository.findById(id).orElse(null);
    }

    @DeleteMapping("/{id}")
```

```
    public void deletar(@PathVariable Long id) {
        repository.deleteById(id);
    }
}
```

Dependências Maven

pom.xml

```
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

</dependencies>
```

Como Executar o Projeto

i. Criar banco de dados MySQL:

```
CREATE DATABASE baozi_store;
```

1 Abrir o projeto na IDE.

2 Executar a classe:

BaoziStoreApplication.java

1 O sistema iniciará em:
http://localhost:8080

1 Testar os endpoints no Postman.

Respostas Esperadas no Postman

POST Cliente


```
{
  "id": 1,
  "nome": "Marcilei4980227",
  "clienteDesde": "2026-05-10"
}
```

POST Produto


```
{
  "id": 1,
  "nome": "Baozi Tradicional de Frango",
  "preco": 12.50,
  "estoque": true
}
```

POST Pedido


```
{
  "id": 1,
  "clienteId": 1,
  "produtoId": 1,
  "quantidade": 3
}
```

Conclusão

O desenvolvimento da API REST da Baozi Store permitiu aplicar os conhecimentos de Java, Spring Boot, Spring Data JPA e arquitetura MVC. O sistema criado realiza o cadastro de clientes, produtos e pedidos, além de permitir consultas, listagens e exclusões através dos endpoints REST.

Os testes realizados no Postman comprovaram o funcionamento correto da aplicação e da integração com o banco de dados relacional.